Blocking Code versus Non-Blocking Code

ECE 2564, Embedded Systems

Fall 2025

Overview

- Blocking versus non-blocking code – What is the issue?
- Timing is important for many embedded applications
- Deadlines matter – can't afford to have code blocked

- How to detect if the code is blocking?
- How to convert blocking code to non-blocking code?

- Import **this** repository
- Let's compile and flash the non-blocking project first

Characteristics of this Application

- Four tasks can be performed at the same time
- One task takes a long time to complete
- Depending on implementation, we can have blocking or non-blocking code

Features of the Non-Blocking Application

- Moving spectrum (sliding on the screen)
- *Pressing* Launchpad S2 lights up the BoosterPack blue LED; releasing S2 turns off the blue LED
- The Launchpad green LED blinks every 1 second
- *Tapping* Launchpad S1 pauses/un-pauses the moving spectrum and toggles Launchpad LED1

Features of the Blocking Application

- Nothing seems to work correctly except for the moving spectrum
- The application is broken
- We will discover where code blocking is taking place

Poster Child Embedded Application

```
int main(void)
{
    // Stop Watchdog Timer
    WDT_A_holdTimer();

    // Initialize the system clock
    InitSystemTiming();

    // Initialize the main Application object and HAL object
    HAL hal = HAL_construct();
    Application app = Application_construct();

    // Main super-loop -- This runs four separate tasks
    while (true)
    {
        Application_loop(&app, &hal);
        HAL_refresh(&hal);
    }
}
```

The First Three Tasks in Application_loop()

```
void Application_loop(Application* app, HAL* hal)
{
    static bool pause = false;
    if (Button_isTapped(&hal->launchpadS1)) {  // TASK 1
        LED_toggle(&hal->launchpadLED1);
        pause = !pause;
    }
    if (Button_isPressed(&hal->launchpadS2)) { // TASK 2
        LED_turnOn(&hal->boosterpackBlue);
    }
    else
    {
        LED_turnOff(&hal->boosterpackBlue);
    }
    if (SWTimer_expired(&app->Launchpad_LED2_blinkingTimer))
    {
        LED_toggle(&hal->launchpadLED2Green);  // TASK 3
        SWTimer_start(&app->Launchpad_LED2_blinkingTimer);
    }
```

Task 4 Spectrum Sweeping: Complex and Time Consuming

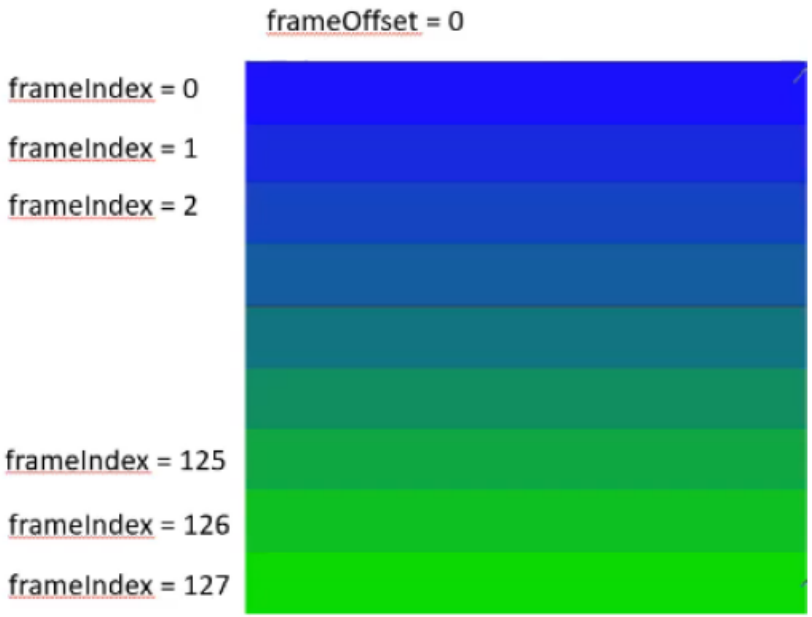
```
unsigned int r, g, b;
    r = 25; // red color is kept constant
    if (!pause) {
        int frameIndex = 0;
        int frameOffset = 0;
        for (frameOffset = 0; frameOffset < 128; frameOffset++)
        {
            for (frameIndex = 0; frameIndex<128; frameIndex++)
            {
                // gradually increase green and reduce blue
                g = frameIndex*2;
                b = 254 - g;
                Graphics_setForegroundColor(&app->gfx.context, colormix(r,g,b));
                Graphics_drawLineH(&app->gfx.context, 0, 127,
                        (frameIndex+frameOffset)%128);
            }
        }
    }
}
```

Why is this Code Blocking?

- Whenver Task 4 is running, the other three tasks cannot be checked or executed
  - Buttons for the first three tasks will not respond when Task 4 is running!
- What is Task 4 doing?
  - Two nested "for" loops
  - It is drawing something on the screen
  - What is it drawing?

Let's Simulate the First Three Iterations of Task 4

Green = 2 * frameIndex
Blue = 254 - Green
y (for horizontal line) = (frameIndex + frameOffset)%128
In this round-> y = frameIndex

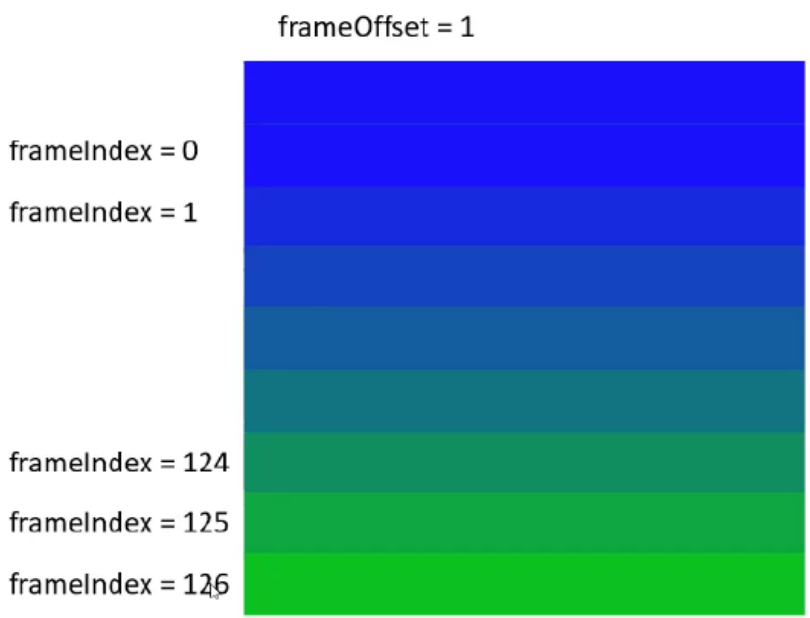


frameOffset = 0

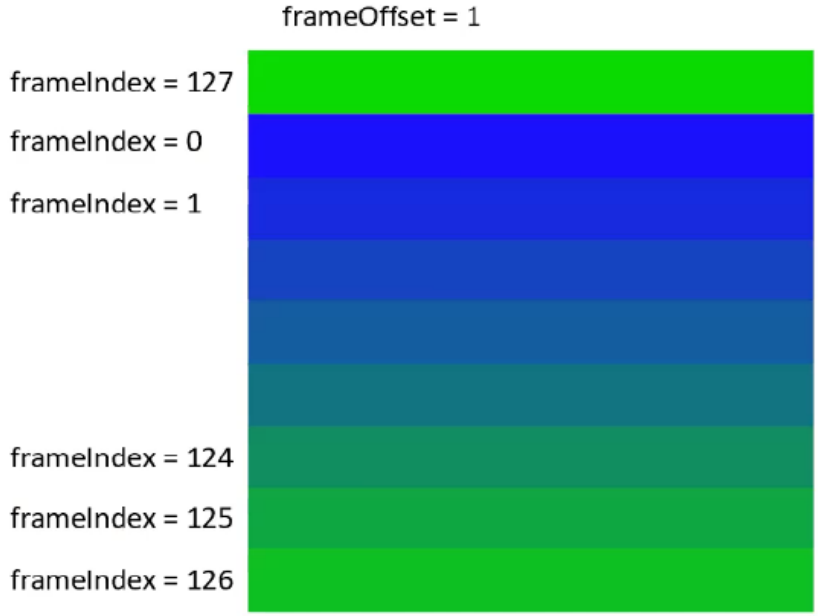
Second iteration: `frameOffset = 1`

- Left image ends at: `frameIndex = 126`
- Right image starts at: `frameIndex = 127`
- The sweeping is scrolling down (and rollover)

Green = 2 * frameIndex
Blue = 254 - Green
y = (frameIndex + frameOffset)%128
In this round -> y = (frameIndex + 1)%128

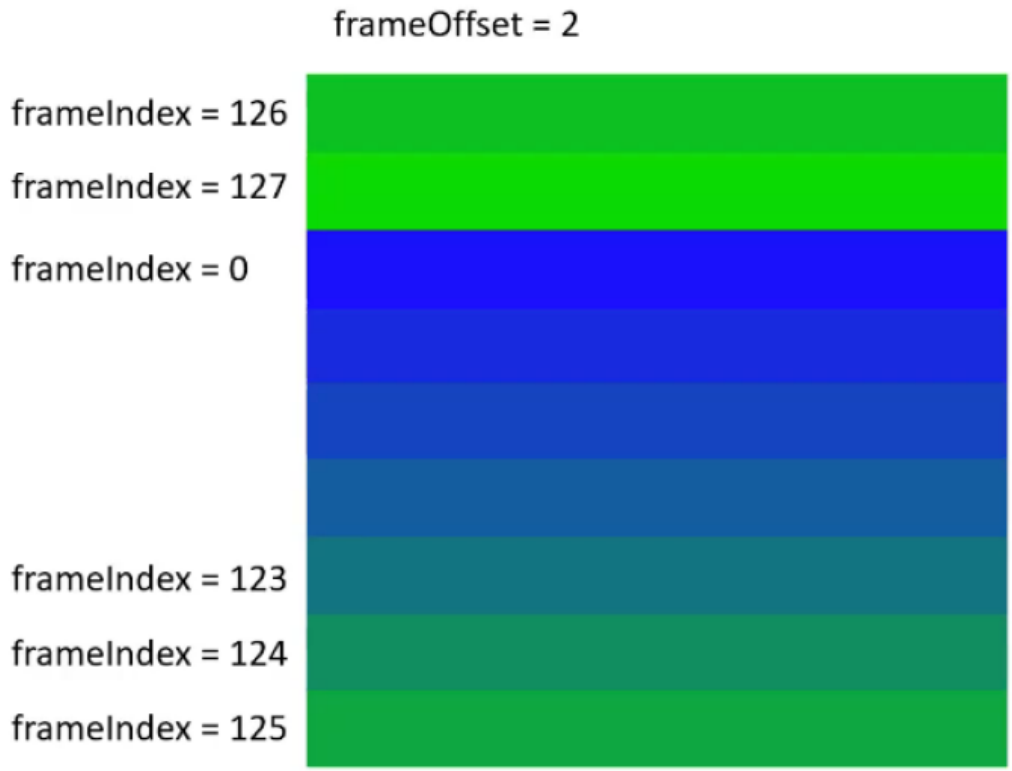


Green = 2 * frameIndex
Blue = 254 - Green
y = (frameIndex + frameOffset)%128
In this round -> y = (frameIndex + 1)%128



Third iteration: `frameOffset = 2`

Green = 2 * frameIndex
Blue = 254 - Green
y_index = (frameIndex + frameOffset)%128



Task 4: Line Drawing Calculation

frameOffset	frameIndex	g (green)	b (blue)	horizontal line's y	Effect
0 to 127	0 to 127	frameIndex*2	254-g	(frameIndex + frameOffset)%128	
0	0	0	254	0	A blue line at the top of the screen
0	63	126	128	63	A blue-green line in the middle of the screen
0	64	128	126	64	A blue-green line in the middle of the screen
0	127	254	0	127	A green line at the bottom of the screen
1	0	0	254	1	A blue line at y=1 (just below the top line)
1	127	254			A green line at the top of the screen
2	0				
120	10	20	234	2	A mostly green line close to the top

Re-write Task 4: Need to Break the "for" loops

```
unsigned int r, g, b;
    r = 25; // red color is kept constant
    if (!pause) {
        int frameIndex = 0;
        int frameOffset = 0;
        for (frameOffset = 0; frameOffset < 128; frameOffset++)
        {
            for (frameIndex = 0; frameIndex<128; frameIndex++)
            {
                // gradually increase green and reduce blue
                g = frameIndex*2;
                b = 254 - g;
                Graphics_setForegroundColor(&app->gfx.context, colormix(r,g,b));
                Graphics_drawLineH(&app->gfx.context, 0, 127,
                        (frameIndex+frameOffset)%128);
            }
        }
    }
}
```

Set up Member Data in the App Constructor

```
  Application Application_construct()
{
    Application app;

    // 1-second timer (i.e. 1000 ms as specified in the SWTimer_contruct)
    app.Launchpad_LED2_blinkingTimer = SWTimer_construct(1000);
    SWTimer_start(&app.Launchpad_LED2_blinkingTimer);

    app.frameIndex = 0;
    app.frameOffset = 0;

    app.gfx = GFX_construct(GRAPHICS_COLOR_WHITE, GRAPHICS_COLOR_BLACK);

    return app;
}
```

Task 4: No Nested Loops – Just Draw a Line

- Use the app member data to keep track of index and offset

```
unsigned int r, g, b;
if (!pause)
{
    r = 25;
    g = app->frameIndex*2;
    b = 254 - g;
    Graphics_setForegroundColor(&app->gfx.context, colormix(r,g,b));
    Graphics_drawLineH(&app->gfx.context, 0, 127, (app->frameIndex+app-
    app->frameIndex++;
    if (app->frameIndex==128)
    {
        app->frameIndex = 0;
        app->frameOffset++;
        if (app->frameOffset==128)
            app->frameOffset = 0;
    }
}
}
```

Summary

- In some applications you need to avoid *blocking* code
- One time-consuming task can block all the other tasks
- Rewrite the complex task so that it will no longer block the other tasks

- How?
  - Analyze the blocking code
  - Identify the bottleneck
  - Find a way to subdivide a time-consuming sub-tasks